

Mohsen Kanj

CSC 343

Professor Donald Lushi

Midterm Project

Spring 2026

Simple ADD/SUB Processor

1- Introduction:

This project uses an instruction language like MIPS to construct a simpler processor that can execute addition and subtraction operations. Data is read from registers, processed by an arithmetic logic unit (ALU), and then written back to a target register.

VHDL was used to implement the design, while Quartus Prime Lite Edition was used for compilation. ModelSim was used for simulation and verification to make sure the system was operating correctly.

2- Instruction Format:

The processor uses a 32-bit R-type instruction format similar to MIPS architecture.

The instruction is divided into the following fields:

- Opcode [31:26] = 000000
- Rd [25:21] = destination register
- Rs [20:16] = source register 1
- Rt [15:11] = source register 2
- shamt [10:6] = 00000
- funct [5:0] = operation selector

The funct field determines the operation:

- 100000 → ADD operation
- 100010 → SUB operation

Additionally:

- Bit 5 of the funct field controls the RegWr signal (register write enable)
- Bit 1 controls the ALU operation (add or subtract)

3- System Design:

The system is composed of several main components that work together to execute add and subtract instructions.

3.1- Register File:

Each of the 32 registers in the register file is 32 bits wide. One writing operation and two reading operations are supported. The instruction's Rs, Rt, and Rd fields are used to access

the registers. The destination register is chosen using a decoder, whereas the source registers are chosen using multiplexers.

3.2- Arithmetic Logic Unit (ALU):

The arithmetic operations of addition and subtraction are carried out by ALU. It generates results based on the ALU control signal after receiving two inputs from the register file (busA and busB).

3.3- Control Unit:

The control unit creates control signals like ALUctr and RegWr by extracting various fields from the instruction. These signals dictate which operation the ALU should carry out and whether a register should be written.

3.4- Multiplexer and Decoder:

Multiplexers are used to route signals within the system, especially for selecting register outputs. A multiplexer is also used to manually initialize register values during simulation, since no memory unit is present. The decoder determines which register will receive the data.

4- Implementation:

VHDL was used to implement the system, with each component designed independently and then connected. The control unit, multiplexers, ALU, and register file are the primary modules.

A structured testbench was used to test each component, which was authored as a single VHDL file. Data can move from the registers to the ALU and back to the register file thanks to the connections between the various parts.

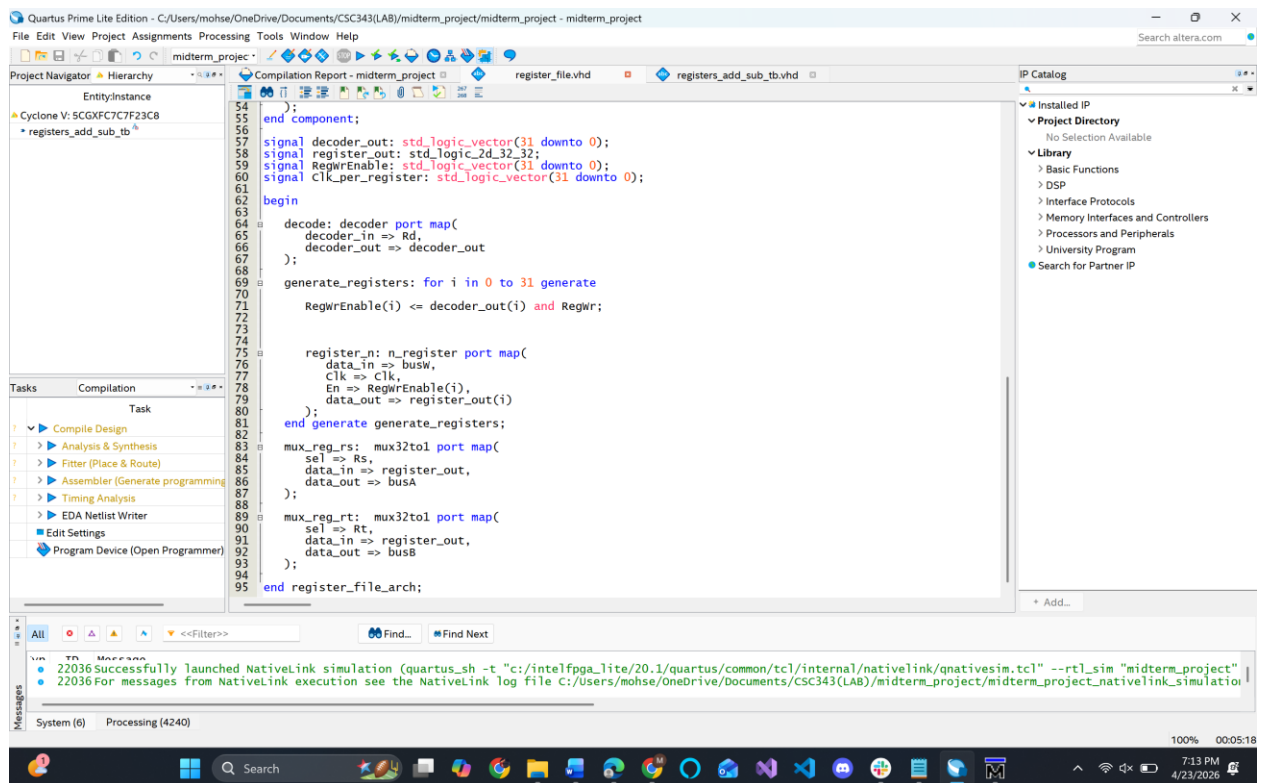


Figure 1: Register file implementation showing 32 registers and write enable logic.

The register file stores 32 registers and allows reading from two registers and writing to one register simultaneously. The decoder selects the destination register, while multiplexers select the source registers.

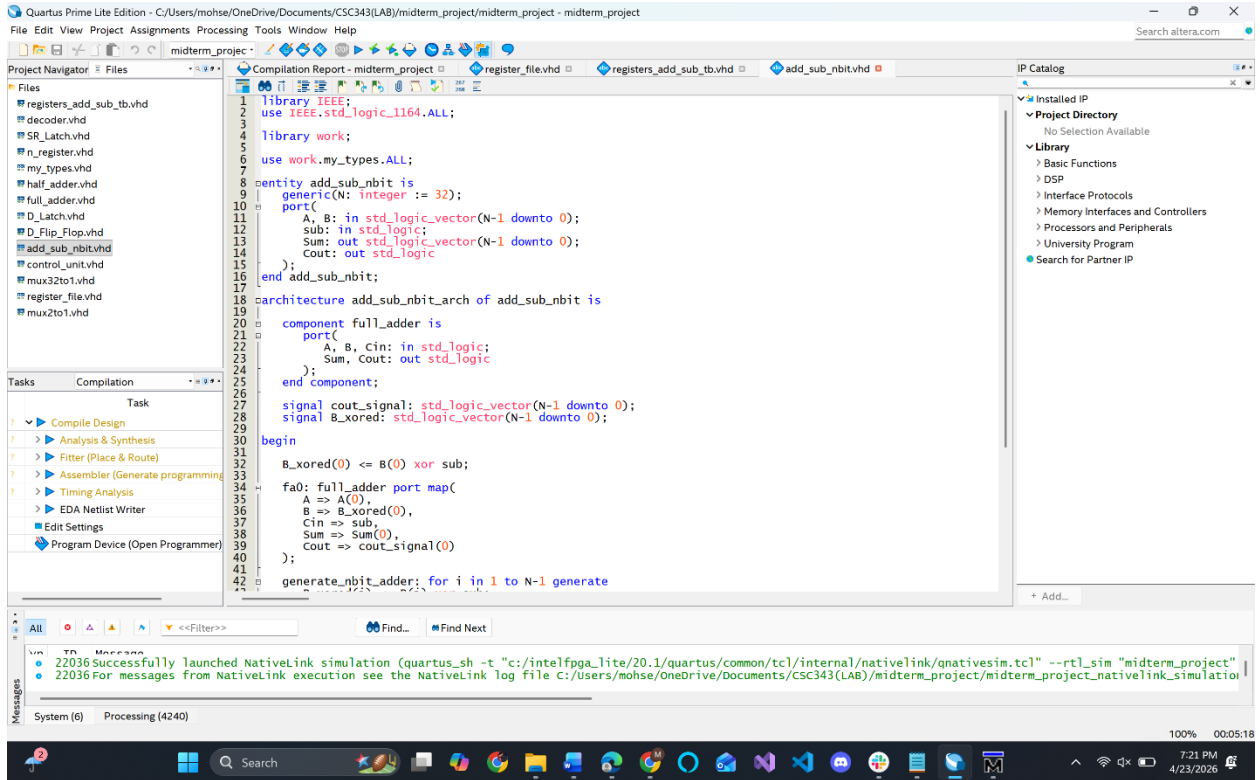


Figure 2: ALU implementation performing addition and subtraction using full adders and the sub control signal.

The ALU performs arithmetic operations based on the control signal. Subtraction is implemented by modifying the input and using full adders.

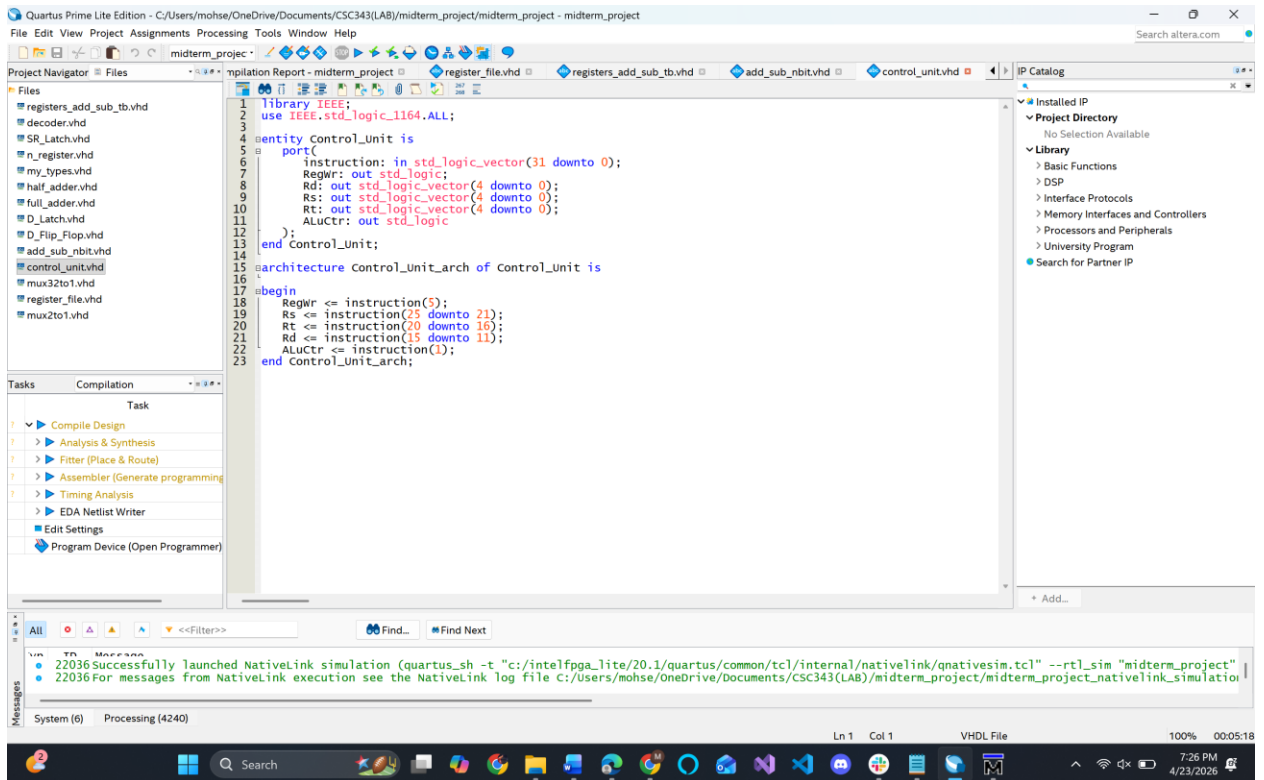


Figure 3: Control unit extracting instruction fields and generating control signals.

The control unit decodes the instruction and generates signals such as RegWr and ALUctr, which control register writing and ALU operations.

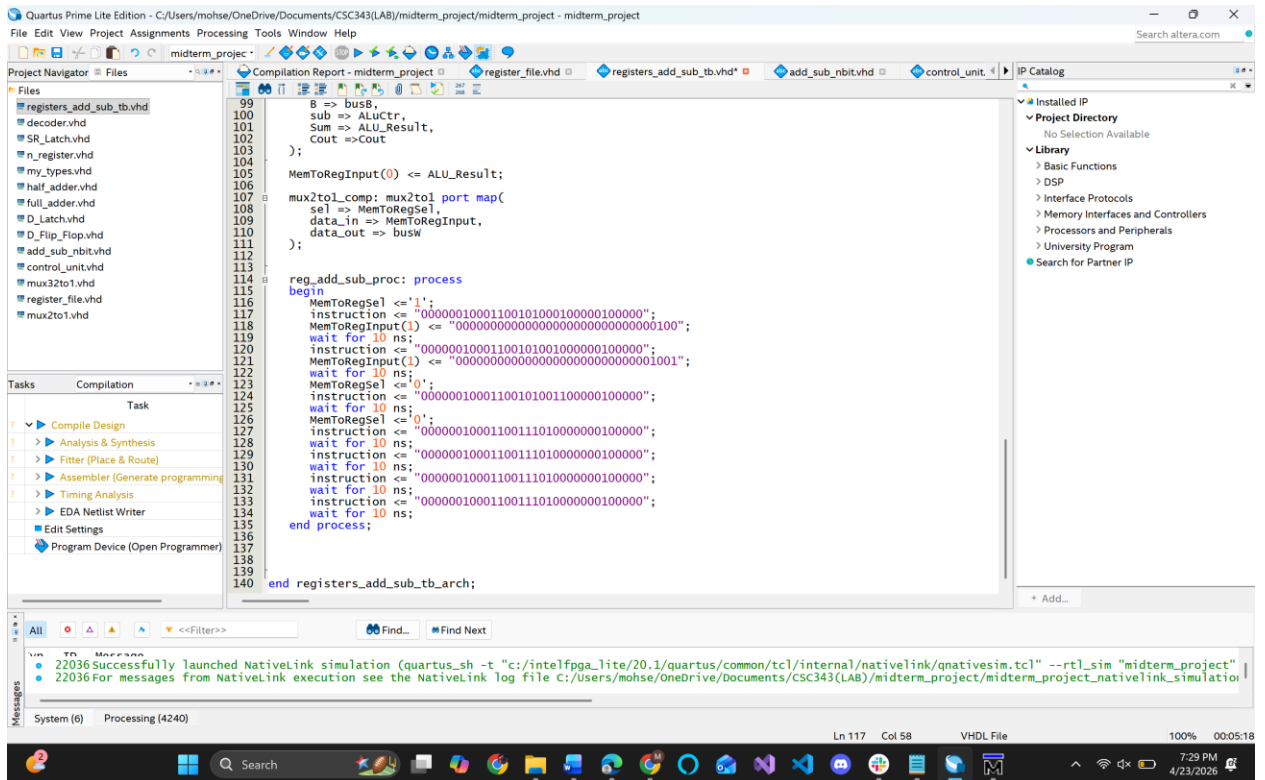


Figure 4: Testbench used to simulate register initialization and instruction execution.

The testbench applies input instructions and initializes registers using a multiplexer. It verifies correct behavior of the system during simulation.

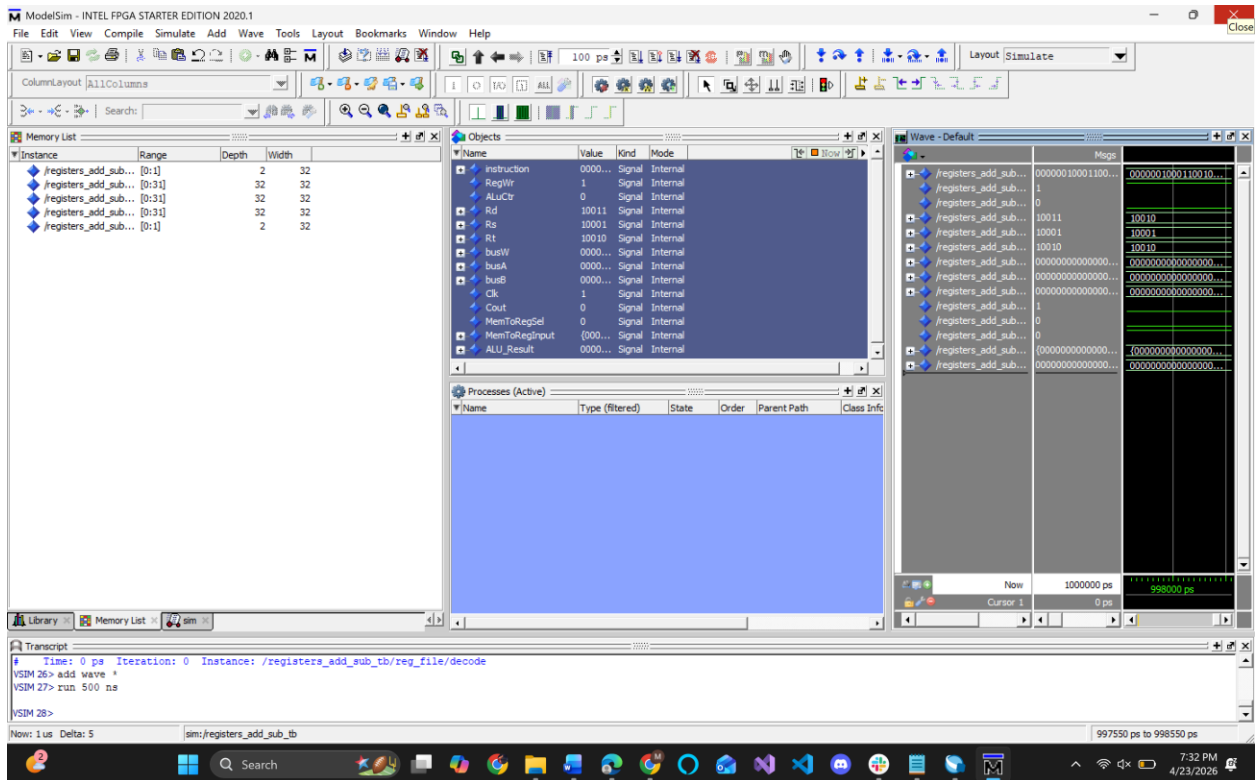


Figure 5: ModelSim simulation environment showing waveform, signal values, and register memory.

This view provides a complete picture of the simulation, including internal signals and register states, confirming correct system operation

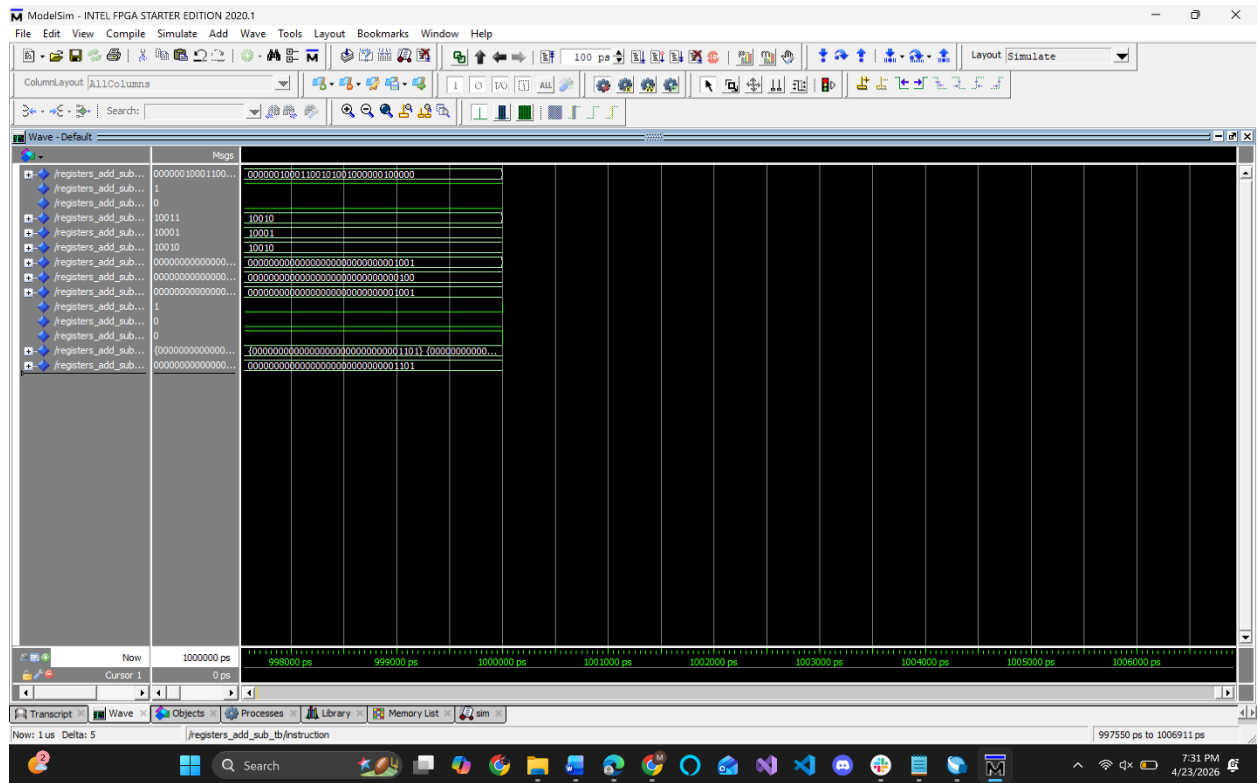


Figure 6: Simulation waveform showing signal transitions over time for instruction execution.

The waveform demonstrates correct timing behavior, where instruction changes result in expected updates in ALU outputs and register values.

Together, Figures 5 and 6 show the general simulation environment as well as the specific time behavior. While the entire view displays how internal signals and register values change throughout execution, the waveform verifies accurate signal transitions.

Conclusion:

This project effectively used VHDL to construct a basic ADD/SUB processor. Important elements like a control unit, register file, multiplexers, and an arithmetic logic unit (ALU) are integrated into the architecture.

The system functions correctly, with instructions successfully interpreted and executed, according to simulation results. Data was appropriately stored and recovered from registers, and the ALU executed addition and subtraction as anticipated.

All things considered, this project gave me hands-on experience with digital system design, VHDL hardware description, and Quartus and ModelSim verification.